ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG LABGUARD

HƯỚNG DẪN THÀNH VIÊN C

ADMIN CLIENT & GIAO DIỆN WINFORMS

Màn hình giám sát phòng máy trực quan, an toàn đa luồng và điều khiển thời gian thực

---

BIÊN SOẠN: VÕ TRỌNG PHÚC

NHÓM THỰC HIỆN: 4 THÀNH VIÊN

TP. HỒ CHÍ MINH — 2026

KHỞI ĐỘNG NHANH DÀNH CHO THÀNH VIÊN C

(Đọc xong trang này trong 2 phút là bạn biết chính xác mình cần làm gì hôm nay)

1. BẠN LÀ AI?

Bạn là người tạo ra ”bộ mặt” trực quan của hệ thống: ứng dụng màn hình quản trị đồ họa (**Admin Client** trên Windows Forms) dành cho người coi thi và quản trị viên phòng máy.

2. BẠN LÀM GÌ?

- Thiết kế giao diện đăng nhập và bảng điều khiển tổng quan phòng máy trực quan, dễ dùng.
- Mở kết nối TCP tới máy chủ Central Server (cổng 5000) để nhận số liệu thời gian thực.
- Hiển thị danh sách máy tính, tô màu cảnh báo xanh/vàng/đỏ khi có máy gặp sự cố.
- Cung cấp các nút bấm để xử lý sự cố (tiếp nhận, giải quyết) và điều khiển máy trạm.

3. BẠN KHÔNG CẦN LO PHẦN NÀO?

- Bạn **không cần** lo việc lấy số liệu phần cứng từ hệ điều hành máy sinh viên (việc của **Thành viên A**).
- Bạn **không cần** lo viết câu lệnh SQL lưu trữ dữ liệu hay tự tính toán nhịp tim sống chết (việc của **Thành viên B**).
- Bạn **không cần** lo phần Gateway/Collector và công cụ thử tải của D.

4. HÔM NAY BẮT ĐẦU Ở ĐÂU?

Mở dự án `src/LabGuard.Admin/` trong Visual Studio, chạy thử file giao diện mẫu `DashboardForm.cs` và thêm các ô nhập tài khoản mật khẩu.

5. FILES BẠN CẦN MỞ TRƯỚC:

1. `START_HERE.md`: Hướng dẫn chạy nhanh cả hệ thống.
2. `src/LabGuard.Admin/DashboardForm.cs`: Màn hình quản trị chính nơi hiển thị bảng số liệu.
3. `src/LabGuard.Admin/Program.cs`: Nơi khởi động ứng dụng giao diện.
4. `src/LabGuard.Protocol/MessageTypes.cs`: Danh mục các loại thông điệp giữa Admin và Central.

MỤC LỤC

1 Tổng quan vai trò của Thành viên C **1**

2 Sáu chặng phát triển ứng dụng Admin Dashboard **2**
2.1 VIỆC TIẾP THEO: Xây dựng bố cục giao diện WinForms trực quan 2
2.2 Chặng 2: VIỆC TIẾP THEO: Mở kết nối TCP tới Central Server 3
2.3 Chặng 3: VIỆC TIẾP THEO: Đăng nhập và phân quyền quản trị (RBAC) 4
2.4 Chặng 4: VIỆC TIẾP THEO: Cập nhật giao diện an toàn đa luồng (BeginInvoke) . 5
2.5 Chặng 5: VIỆC TIẾP THEO: Xử lý xung đột Khóa lạc quan (OCC) trên màn hình 6
2.6 VIỆC TIẾP THEO: Kiểm thử thủ công và hoàn thiện giao diện 7

3 Năm bẫy lỗi lập trình cần tránh dành cho Thành viên C **7**

4 Bộ câu hỏi vấn đáp dành riêng cho Thành viên C **9**

1. Tổng quan vai trò của Thành viên C

TÓM TẮT 30 GIÂY

- Bạn phụ trách ”bộ mặt” trực quan của đồ án: ứng dụng quản trị WinForms (`LabGuard.Admin`).
- Kết nối TCP tới cổng 5000 của Central để nhận số liệu và gửi lệnh điều khiển.
- Xử lý an toàn đa luồng (`BeginInvoke`) để không bao giờ làm đơ chuột hay văng ứng dụng.
- Hiển thị cảnh báo thông minh khi có hai người cùng thao tác trên một sự cố (OCC).

Trong toàn bộ đồ án LabGuard, bạn là người chịu trách nhiệm về ”bộ mặt” của sản phẩm. Khi giảng viên chấm thi, điều đầu tiên thầy cô nhìn thấy và trực tiếp trải nghiệm chính là ứng dụng giao diện WinForms do bạn thiết kế:

KHÁI NIỆM MẠNG CỐT LÕI

Trọng trách cốt lõi của Thành viên C:

1. **Thiết kế giao diện WinForms (`src/LabGuard.Admin/`):** Xây dựng bảng điều khiển `DashboardForm.cs` trực quan, hiển thị sơ đồ phòng máy và danh sách sự cố.
2. **Lập trình kết nối mạng TCP Client:** Kết nối ứng dụng WinForms tới cổng 5000 của Central Server để nhận các thông báo sự kiện theo thời gian thực.
3. **Xử lý an toàn đa luồng (Cross-Thread UI):** Chuyển các gói tin nhận từ luồng mạng ngầm về luồng giao diện chính một cách an toàn bằng `BeginInvoke`.
4. **Giải quyết xung đột OCC:** Hiển thị cảnh báo thông minh khi có hai quản trị viên cùng thao tác trên một sự cố.

Hình này chỉ cần nhìn vào hai hộp bên trên và chiếc cầu nối bên dưới: luồng giao diện và luồng mạng trao đổi an toàn qua lệnh `BeginInvoke`.

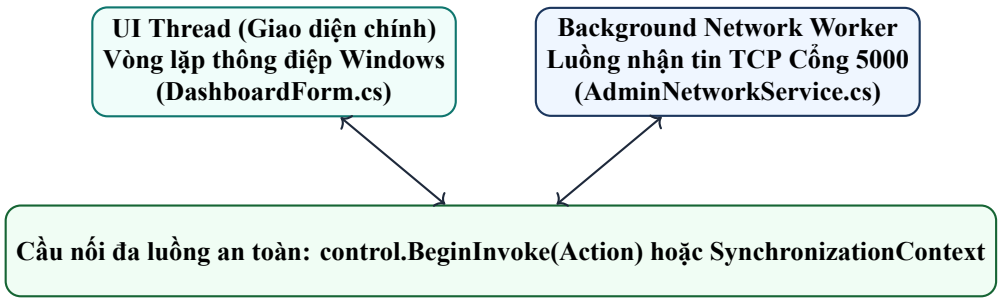


Figure 1: Mô hình phân tách giữa luồng giao diện UI và luồng mạng nền trong WinForms

Ý chính: Mọi thao tác cập nhật dữ liệu lên giao diện đồ họa bắt buộc phải được điều phối qua luồng giao diện chính để ứng dụng không bị văng lỗi.

GHI CHÚ QUAN TRỌNG

Hiện trạng mã nguồn thực tế: Toàn bộ phần việc của Thành viên C đang ở trạng thái **CHƯA LÀM / DỰ KIẾN**. Thư mục `src/LabGuard.Admin/` hiện chỉ có file `Program.cs` và form

giao diện rỗng. Bạn sẽ tự tay thiết kế và lập trình theo lộ trình 6 chặng dưới đây.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Luôn tách biệt rõ ràng giữa luồng mạng (đọc socket) và luồng giao diện (vẽ màn hình).
- Mọi gói tin gửi lên Central trên cổng 5000 đều được đóng khung 4 byte qua `FrameCodec`.

2. Sáu chặng phát triển ứng dụng Admin Dashboard

TÓM TẮT 30 GIÂY

- Chặng 1: Dựng layout trực quan (lưới phòng máy, danh sách sự cố, nút thao tác).
- Chặng 2–3: Mở kết nối TCP Cổng 5000 và màn hình đăng nhập phân quyền RBAC.
- Chặng 4: Cập nhật giao diện an toàn đa luồng bằng `BeginInvoke`.
- Chặng 5–6: Xử lý xung đột khóa lạc quan OCC và hoàn thiện trải nghiệm người dùng.

2.1 VIỆC TIẾP THEO: Xây dựng bố cục giao diện WinForms trực quan

Chặng 1 — Mã tác vụ nội bộ: LG-C-001, LG-C-002

2.1.1 Tại sao cần phần này?

Người quản trị hoặc giảng viên coi thi không thể đọc từng dòng chữ đen trắng của 50 máy tính. Họ cần một bảng điều khiển đồ họa: nhìn lướt qua là biết ngay máy nào đang tốt (xanh), máy nào tải cao (vàng), máy nào bị tắt (đỏ).

2.1.2 Khi chạy, chuyện gì xảy ra?

Bố trí 4 khu vực giao diện rõ ràng bằng các Control chuẩn WinForms:

1. **Thanh trên cùng:** Hiển thị trạng thái kết nối mạng, địa chỉ IP:5000 và nút Đăng nhập.
2. **Lưới phòng máy (DataGridView):** Hiển thị danh sách máy tính, mức CPU/RAM và tô màu xanh/vàng/đỏ theo trạng thái.
3. **Bảng sự cố thời gian thực:** Hiển thị các sự cố phát sinh (`HIGH_CPU`, `DEVICE_OFFLINE`).
4. **Khu vực nút bấm:** Nút ”Tiếp nhận sự cố”, ”Giải quyết xong” và ”Khởi động lại máy”.

2.1.3 Bạn sẽ đụng file nào?

Sửa file: `src/LabGuard.Admin/DashboardForm.cs` và `DashboardForm.Designer.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Sự kiện `CellFormatting` tự động chạy khi vẽ từng ô dữ liệu.
- Tô màu nền và màu chữ tương ứng với trạng thái ONLINE (xanh), WARNING (vàng), OFFLINE (đỏ).

```
// To mau dong tu dong theo trang thai may tinh
private void dgvDevices_CellFormatting(object sender,
    DataGridViewCellFormattingEventArgs e)
{
    if (e.RowIndex < 0 || e.RowIndex >= dgvDevices.Rows.Count) return;
    string status = dgvDevices.Rows[e.RowIndex].Cells["Status"].Value?.ToString()
        ?? "OFFLINE";
    var row = dgvDevices.Rows[e.RowIndex];
    if (status == "ONLINE") row.DefaultCellStyle.BackColor = Color.FromArgb(240,
        253, 244);
    else if (status == "WARNING") row.DefaultCellStyle.BackColor =
        Color.FromArgb(254, 252, 232);
    else row.DefaultCellStyle.BackColor = Color.FromArgb(254, 242, 242);
}
```

Listing 1: Ví dụ rút gọn: Tô màu hàng theo trạng thái trong DataGridView

2.1.4 Làm sao thử?

Bấm phím `F5` trong Visual Studio: cửa sổ Dashboard xuất hiện với đầy đủ bảng lưới, màu sắc hài hòa và các nút bấm rõ ràng.

2.1.5 Khi nào coi là xong? (PASS Criteria)

- Giao diện hiển thị đầy đủ 4 khu vực chức năng.
- Lưới dữ liệu tự động đổi màu chuẩn xác khi trạng thái máy thay đổi.

2.2 Chặng 2: VIỆC TIẾP THEO: Mở kết nối TCP tới Central Server

Mã tác vụ nội bộ: LG-C-003

2.2.1 Tại sao cần phần này?

Dashboard không thể tự sinh ra dữ liệu. Nó phải mở một luồng socket TCP kết nối tới Central Server trên cổng 5000 để nhận danh sách máy tính và hứng các thông báo sự cố phát sinh.

2.2.2 Khi chạy, chuyện gì xảy ra?

Khi bấm ”Kết nối”, Admin mở `TcpClient` kết nối tới Central cổng 5000, sau đó chạy vòng lặp ngầm đọc từng khung tin bằng `FrameCodec.ReadFrameAsync` và phát sự kiện `OnMessageReceived`.

2.2.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Admin/AdminNetworkService.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Khởi tạo `TcpClient` kết nối tới host và port 5000.
- Chạy vòng lặp ngầm nhận tin qua `FrameCodec` mà không làm đơ giao diện đồ họa.

```
// Pseudocode minh hoa: Doc khung tin ngam tu Central
public async Task StartReceiveLoopAsync(NetworkStream stream, CancellationToken
    ct)
{
    while (!ct.IsCancellationRequested)
    {
        byte[]? frame = await FrameCodec.ReadFrameAsync(stream, ct: ct);
        if (frame == null) break;
        var env = ProtocolSerializer.DeserializeEnvelope(frame);
        if (env != null) OnMessageReceived?.Invoke(env);
    }
}
```

Listing 2: Pseudocode minh họa: Vòng lặp nhận tin ngầm của Admin

2.2.4 Làm sao thử?

Bật Central lên, mở Dashboard bấm ”Kết nối”: thanh trạng thái chuyển sang màu xanh lá ”Đã kết nối”.

2.2.5 Khi nào coi là xong? (PASS Criteria)

- Kết nối TCP cổng 5000 ổn định, không làm giật lag giao diện.
- Bóc tách chính xác từng khung tin qua `FrameCodec`.

2.3 Chặng 3: VIỆC TIẾP THEO: Đăng nhập và phân quyền quản trị (RBAC)

Mã tác vụ nội bộ: LG-C-004

2.3.1 Tại sao cần phần này?

Không phải ai mở Dashboard lên cũng được phép bấm nút khởi động lại máy tính sinh viên hay đóng sự cố. Cần có màn hình đăng nhập để phân chia quyền: `Admin` (toàn quyền) và `Viewer` (chỉ được xem).

2.3.2 Khi chạy, chuyện gì xảy ra?

Admin nhập tài khoản mật khẩu vào `LoginForm.cs`. Giao diện gửi gói `LOGIN` lên cổng 5000. Central trả về `LOGIN_RESULT` chứa vai trò (`Role`). Nếu là `Viewer`, các nút bấm thao tác nguy hiểm sẽ bị mờ đi (`Enabled = false`).

2.3.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Admin/LoginForm.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Kiểm tra `result.Role`: nếu là Viewer thì khóa các nút bấm điều khiển.
- Hiển thị tên người dùng và quyền hạn rõ ràng trên thanh tiêu đề.

```
// Phan quyen nut bam sau khi dang nhap thanh cong
private void ApplyRolePermissions(string role)
{
    bool isAdmin = (role == "Admin");
    btnAcknowledge.Enabled = isAdmin;
    btnResolve.Enabled = isAdmin;
    btnReboot.Enabled = isAdmin;
}
```

Listing 3: Ví dụ rút gọn: Phân quyền vai trò sau đăng nhập

2.3.4 Làm sao thử?

Đăng nhập tài khoản Viewer: các nút bấm bị mờ đi. Đăng nhập tài khoản Admin: các nút bấm sáng lên và thao tác được bình thường.

2.3.5 Khi nào coi là xong? (PASS Criteria)

- Phân quyền chính xác giữa Admin và Viewer.
- Khóa an toàn các nút bấm khi chưa đủ thẩm quyền.

2.4 Chặng 4: VIỆC TIẾP THEO: Cập nhật giao diện an toàn đa luồng (BeginInvoke)

Mã tác vụ nội bộ: LG-C-005

2.4.1 Tại sao cần phần này?

Đây là bẫy lỗi số 1 của sinh viên làm đồ án WinForms! Luồng đọc socket chạy ngầm trong hệ thống. Nếu từ luồng này bạn cập nhật trực tiếp Control trên form, WinForms sẽ lập tức bị văng ngoại lệ `Cross-thread operation not valid` và tắt ngúm!

2.4.2 Khi chạy, chuyện gì xảy ra?

Mọi Control trong WinForms chỉ được phép truy cập từ luồng tạo ra nó (UI Thread). Để chuyển dữ liệu từ luồng socket sang UI Thread an toàn, bạn kiểm tra `InvokeRequired` và gọi `BeginInvoke`:

2.4.3 Bạn sẽ đụng file nào?

Sửa file: `src/LabGuard.Admin/DashboardForm.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Kiểm tra `this.InvokeRequired` để biết có đang ở luồng mạng hay không.
- Dùng `BeginInvoke` chuyển lời gọi hàm về UI Thread mà không làm nghẽn luồng mạng.

```
// Cap nhat luoi du lieu an toan da luong bang BeginInvoke
public void UpdateDeviceGridSafe(string deviceId, double cpu, string status)
{
    if (this.InvokeRequired)
    {
```

```
        this.BeginInvoke(new Action(() => UpdateDeviceGridSafe(deviceId, cpu,
            status)));
        return;
    }
    // Da o UI Thread an toan: cap nhat truc tiep len DataGridView
    UpdateRowCells(deviceId, cpu, status);
}
```

Listing 4: Ví dụ rút gọn: Cập nhật giao diện an toàn đa luồng

2.4.4 Làm sao thử?

Cho Central liên tục bắn các gói thông báo sự cố: bảng lưới trên màn hình cập nhật liên tục mà con trỏ chuột vẫn cuộn lướt mượt mà, không bị văng lỗi.

2.4.5 Khi nào coi là xong? (PASS Criteria)

- 100% các thao tác cập nhật giao diện đều đi qua `BeginInvoke`.
- Tuyệt đối không xảy ra ngoại lệ `InvalidOperationException`.

2.5 Chặng 5: VIỆC TIẾP THEO: Xử lý xung đột Khóa lạc quan (OCC) trên màn hình

Mã tác vụ nội bộ: LG-C-006

2.5.1 Tại sao cần phần này?

Nếu hai Admin cùng nhìn thấy một sự cố và cùng bấm nút ”Tiếp nhận”, người bấm sau sẽ nhận phản hồi lỗi `409 Conflict` từ Central Server. Giao diện phải thông báo khéo léo để người đó hiểu sự cố đã được đồng nghiệp xử lý trước.

2.5.2 Khi chạy, chuyện gì xảy ra?

Khi bấm nút ”Tiếp nhận sự cố”, Dashboard gửi kèm `ExpectedRowVersion`.

- Nếu kết quả là `200 OK`: Đổi màu dòng sự cố sang màu đã tiếp nhận (`ACKNOWLEDGED`).
- Nếu kết quả là `409 Conflict`: Hiện hộp thoại: *”Sự cố này đã được quản trị viên khác tiếp nhận trước!”* và tự động tải lại dữ liệu mới nhất.

2.5.3 Bạn sẽ đụng file nào?

Sửa file: `src/LabGuard.Admin/DashboardForm.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Bắt mã trạng thái 409 Conflict từ máy chủ.
- Hiển thị thông báo thân thiện và tự động làm mới danh sách dữ liệu.

```
// Xu ly phan hoi OCC khi tiep nhan su co
var result = await _networkService.SendCommandAsync(cmd);
if (result.StatusCode == 200)
{
```

```
    MessageBox.Show("Tiep nhan su co thanh cong!");
}
else if (result.StatusCode == 409)
{
    MessageBox.Show("Ban ghi da duoc Admin khac tiep nhan truoc!", "Canh bao
        OCC");
    await RefreshDataAsync(); // Tu dong tai lai du lieu moi
}
```

Listing 5: Ví dụ rút gọn: Xử lý phản hồi 409 Conflict trên giao diện

2.5.4 Khi nào coi là xong? (PASS Criteria)

- Bắt đúng mã phản hồi 409 Conflict từ máy chủ.
- Tự động làm mới giao diện khi có xung đột dữ liệu.

2.6 VIỆC TIẾP THEO: Kiểm thử thủ công và hoàn thiện giao diện

Chặng 6 — Mã tác vụ nội bộ: LG-C-007

2.6.1 Tại sao cần phần này?

Màn hình quản trị là nơi hiển thị trực tiếp dữ liệu hoạt động của hệ thống. Giao diện trực quan, ổn định, xử lý lỗi thân thiện sẽ nâng cao chất lượng trải nghiệm của đồ án.

2.6.2 Khi chạy, chuyện gì xảy ra?

Thực hiện kịch bản kiểm thử toàn diện: đăng nhập tài khoản, mất mạng rồi kết nối lại, tiếp nhận sự cố, lọc danh sách máy theo trạng thái và gửi lệnh điều khiển.

2.6.3 Khi nào coi là xong? (PASS Criteria)

- Mọi luồng tương tác của người dùng diễn ra trơn tru.
- Giao diện không bị giật lag ngay cả khi nhận dữ liệu dồn dập.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Mọi cập nhật UI từ luồng mạng phải dùng `BeginInvoke`.
- Luôn xử lý mã lỗi 409 Conflict một cách lịch sự và tải lại dữ liệu mới.

3. Năm bẫy lỗi lập trình cần tránh dành cho Thành viên C

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 1: Cập nhật Control từ luồng mạng ngầm (Cross-thread Exception)
Không bao giờ gán trực tiếp thuộc tính Control từ luồng nhận tin của Socket. Luôn kiểm tra `InvokeRequired` và gọi `BeginInvoke`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Gán trực tiếp từ luồng nền sẽ ném ngoại lệ làm sập ứng dụng.
- Dùng `BeginInvoke` chuyển việc về cho UI Thread thực thi.

```
// [SAI] Vang loi System.InvalidOperationException:
lblStatus.Text = "Connecting..."; // Goi tu ThreadPool Thread

// [DUNG] Chuyen ve UI Thread an toan bang BeginInvoke:
if (lblStatus.InvokeRequired)
    lblStatus.BeginInvoke(new Action(() => lblStatus.Text = "Connecting..."));
else
    lblStatus.Text = "Connecting...";
```

Listing 6: So sánh Bẫy 1: Gán trực tiếp vs BeginInvoke chuẩn

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 2: Dùng Invoke đồng bộ thay vì BeginInvoke bất đồng bộ
Gọi `control.Invoke` sẽ bắt luồng mạng ngầm phải đứng chờ luồng giao diện vẽ xong mới được đi tiếp. Nếu giao diện đang bận vẽ, luồng mạng sẽ bị nghẽn làm chậm các gói tin tiếp theo. Luôn ưu tiên dùng `BeginInvoke`.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 3: Đóng băng giao diện do gọi lệnh mạng đồng bộ
Nếu bạn viết `client.Connect("127.0.0.1", 5000)` trực tiếp trong sự kiện nút bấm `btnConnect_Click` mà không dùng `await`, con trỏ chuột sẽ biến thành hình vòng xoay và giao diện bị đơ cứng (*Not Responding*). Luôn dùng `async/await`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Lệnh mạng đồng bộ khóa cứng vòng lặp vẽ giao diện.
- `await client.ConnectAsync` giúp giao diện vẫn phản hồi chuột trong lúc chờ.

```
// [SAI] Dong bang giao dien chuot quay vong xoay (Not Responding):
private void btnConnect_Click(object sender, EventArgs e) { client.Connect(ip,
    port); }

// [DUNG] Giao dien van muot ma trong luc cho ket noi:
private async void btnConnect_Click(object sender, EventArgs e) { await
    client.ConnectAsync(ip, port); }
```

Listing 7: So sánh Bẫy 3: Blocking Connect vs Async Connect

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 4: Quên hủy đăng ký sự kiện gây rò rỉ bộ nhớ (Memory Leak)
Nếu form phụ đăng ký lắng nghe sự kiện từ `AdminNetworkService` mà khi đóng form không hủy đăng ký (`-=`), form phụ đó sẽ không bao giờ được bộ dọn rác (GC) thu hồi.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 5: Không khóa các nút bấm khi đang chờ máy chủ phản hồi
Khi bấm nút "Khởi động lại máy", nếu không tạm thời vô hiệu hóa nút bấm (`Enabled = false`), người dùng có thể bấm liên tục 5–10 lần khiến máy chủ nhận dồn dập nhiều lệnh trùng lặp.

4. Bộ câu hỏi vấn đáp dành riêng cho Thành viên C

Dưới đây là 20 câu hỏi trọng tâm thường gặp khi bảo vệ đồ án phần giao diện người dùng:

Câu 1: Lỗi Cross-thread trên WinForms là gì? Tại sao lại xảy ra?
Trả lời: Là ngoại lệ `InvalidOperationException` xảy ra khi một luồng nền (ví dụ luồng nhận tin mạng TCP) cố gắng truy cập trực tiếp vào thuộc tính của một điều khiển giao diện. WinForms quy định chỉ luồng tạo ra điều khiển (UI Thread) mới có quyền thao tác trực tiếp trên nó.

Câu 2: Khắc phục lỗi Cross-thread như thế nào?
Trả lời: Khắc phục bằng cách kiểm tra thuộc tính `InvokeRequired`. Nếu trả về `true`, ta dùng phương thức `BeginInvoke` để đẩy hành động sang cho UI Thread thực hiện.

Câu 3: Tại sao em chọn BeginInvoke thay vì Invoke thông thường?
Trả lời: `Invoke` là gọi đồng bộ, luồng mạng sẽ bị chặn đứng chờ UI vẽ xong mới chạy tiếp, dễ làm chậm các gói tin mạng tiếp theo. `BeginInvoke` là gọi bất đồng bộ, đẩy tác vụ vào hàng đợi thông điệp của UI rồi luồng mạng tiếp tục đọc gói tin ngay lập tức, giúp ứng dụng mượt mà.

Câu 4: Admin Client kết nối tới cổng nào của Central Server?
Trả lời: Kết nối tới cổng 5000 (Central TCP Endpoint), điểm tiếp nhận kết nối chung duy nhất của hệ thống. Central Server dựa vào bản tin đăng nhập `LOGIN` ban đầu để phân loại đối tác vào kênh quản trị.

Câu 5: Giao diện của em có những khu vực chính nào?
Trả lời: Gồm 4 khu vực: 1. Thanh trạng thái kết nối và đăng nhập; 2. Bảng lưới danh sách máy trạm phòng thi; 3. Bảng danh sách sự cố thời gian thực; 4. Khu vực các nút bấm điều khiển sự cố và máy trạm.

Câu 6: Màu sắc trên bảng lưới máy trạm biểu thị điều gì?
Trả lời: Màu xanh lá: máy trạm trực tuyến bình thường; Màu vàng cam: máy đang có cảnh báo tài nguyên; Màu đỏ: máy bị mất mạng hoặc đang gặp sự cố nghiêm trọng.

Câu 7: Hệ thống của em phân quyền quản trị như thế nào?
Trả lời: Phân chia thành 2 vai trò qua cơ chế RBAC: vai trò `Admin` có toàn quyền xử lý sự cố và gửi lệnh điều khiển; vai trò `Viewer` chỉ có quyền xem thông số, các nút bấm thao tác sẽ bị vô hiệu hóa.

Câu 8: Khi hai Admin cùng bấm tiếp nhận một sự cố, giao diện xử lý thế nào?
Trả lời: Dashboard gửi kèm số phiên bản `ExpectedRowVersion`. Người bấm trước được máy chủ trả về `200 OK`. Người bấm sau nhận mã lỗi `409 Conflict`, giao diện sẽ hiện hộp thoại cảnh báo và tự động tải lại trạng thái mới nhất của sự cố.

Câu 9: Làm thế nào để giao diện không bị đơ chuột khi đang kết nối mạng?
Trả lời: Bằng cách sử dụng từ khóa `async/await` với phương thức `ConnectAsync`, giúp

giải phóng luồng giao diện để tiếp tục xử lý các sự kiện chuột và bàn phím trong thời gian chờ bắt tay mạng.

Câu 10: Khi mở form lên lần đầu, Dashboard lấy danh sách máy tính bằng cách nào?
Trả lời: Gửi bản tin yêu cầu `HELLO_ADMIN` lên Central Server; Central Server sẽ truy vấn CSDL và gửi trả danh sách thiết bị hiện có.

Câu 11: Sự cố có những trạng thái nào trên bảng giao diện?
Trả lời: 3 trạng thái chuẩn: `OPEN` (Mới phát hiện), `ACKNOWLEDGED` (Đã tiếp nhận xử lý), và `RESOLVED` (Đã giải quyết xong).

Câu 12: Người quản trị có thể gửi lệnh điều khiển gì xuống máy tính?
Trả lời: Có thể gửi lệnh điều khiển từ xa (khởi động lại máy, khóa màn hình) thông qua các bản tin quản trị chuẩn hóa trong giao thức, và nhận kết quả qua bản tin `COMMAND_RESULT`.

Câu 13: Tại sao em không dùng GDI+ để tự vẽ biểu đồ?
Trả lời: Để giữ mã nguồn đơn giản, dễ bảo trì và tập trung vào kiến trúc mạng cốt lõi theo đúng yêu cầu đồ án môn Lập trình mạng.

Câu 14: Dữ liệu gửi từ Admin lên Central có được đóng khung 4 byte không?
Trả lời: Có, toàn bộ kết nối TCP trong LabGuard đều tuân thủ nghiêm ngặt chuẩn đóng khung 4 byte Big-Endian của `FrameCodec`.

Câu 15: Nếu giảng viên yêu cầu đăng xuất để đổi tài khoản khác thì form xử lý thế nào?
Trả lời: Bấm nút Đăng xuất, ứng dụng xóa thông tin phiên làm việc, khóa lại các chức năng quản trị và hiển thị lại hộp thoại Đăng nhập.

Câu 16: Thành viên C phối hợp với Thành viên B như thế nào khi làm đồ án?
Trả lời: Hai bạn thống nhất các loại thông điệp quản trị trao đổi trên cổng 5000 (`LOGIN`, `LOGIN_RESULT`, `INCIDENT_ACK`, `COMMAND_RESULT`) dựa trên hợp đồng giao thức chuẩn `LabGuard.Protocol`.

Câu 17: Làm thế nào để người quản trị biết kết nối mạng giữa Dashboard và Server bị đứt?
Trả lời: Khi luồng mạng ngầm ném ngoại lệ mất kết nối, dịch vụ kích hoạt sự kiện cập nhật giao diện: thanh trạng thái đổi sang chữ ”MẤT KẾT NỐI” màu đỏ và tự động thử kết nối lại.

Câu 18: Em dùng công cụ gì để thiết kế giao diện WinForms?
Trả lời: Sử dụng Visual Studio Windows Forms Designer kết hợp với việc tùy biến màu sắc và sự kiện `CellFormatting` trong mã nguồn C#.

Câu 19: Hiện tại mã nguồn của Admin Dashboard đã làm được những gì?
Trả lời: Hiện tại mã nguồn Admin đang ở giai đoạn phân công nhiệm vụ (CHƯA LÀM / DỰ KIẾN). Em sẽ tự tay thiết kế và lập trình hoàn chỉnh phân hệ này.

Câu 20: Điểm sáng tạo nhất trong phần việc của em là gì?
Trả lời: Là việc xử lý xung đột khóa lạc quan (OCC) trực quan ngay trên giao diện và tách biệt hoàn toàn giữa luồng giao diện UI và luồng mạng nền, giúp ứng dụng luôn phản hồi tức thời mà không bị giật lag.

— HẾT CẨM NANG HƯỚNG DẪN THÀNH VIÊN C —